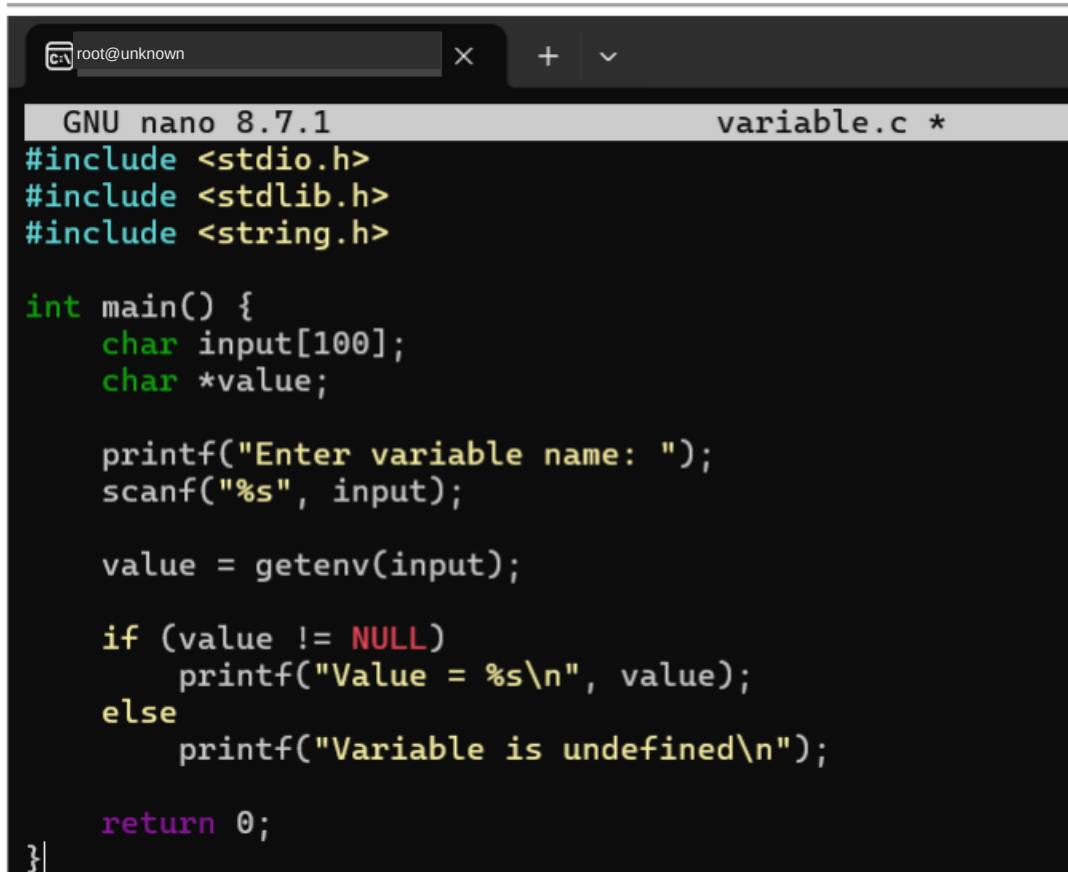


(SKILL Content Delivery Concepts)

1. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.



```
GNU nano 8.7.1 variable.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char input[100];
    char *value;

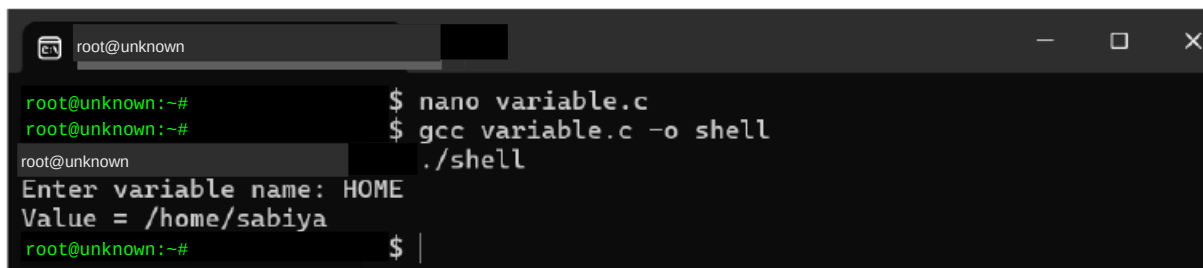
    printf("Enter variable name: ");
    scanf("%s", input);

    value = getenv(input);

    if (value != NULL)
        printf("Value = %s\n", value);
    else
        printf("Variable is undefined\n");

    return 0;
}
```

OUTPUT SCREENSHOT:



```
root@unknown:~# $ nano variable.c
root@unknown:~# $ gcc variable.c -o shell
root@unknown:~# ./shell
Enter variable name: HOME
Value = /home/sabiya
root@unknown:~# $
```

2. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```
GNU nano 8.7.1 cd.c *
#include <stdio.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char path[PATH_MAX];

    printf("Current directory:\n");
    getcwd(path, sizeof(path));
    printf("%s\n", path);

    if (chdir("/tmp") == 0) {
        printf("Directory changed successfully\n");

        getcwd(path, sizeof(path));
        printf("New directory: %s\n", path);
    } else {
        printf("Failed to change directory\n");
    }

    return 0;
}
```

OUTPUT SCREENSHOT:

```
root@unknown
root@unknown:~# nano variable.c
root@unknown:~# $ gcc variable.c -o shell
root@unknown:~# $ ./shell
Enter variable name: HOME
Value = /home/sabiya
root@unknown:~# $ nano cd.c
root@unknown:~# $ gcc cd.c -o shell
root@unknown:~# $ ./shell
Current directory:
/home/sabiya
Directory changed successfully
New directory: /tmp
root@unknown:~# $ |
```

3. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```
root@unknown
GNU nano 8.7.1 directory.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    char path[100];

    if (getcwd(path, sizeof(path)) != NULL)
        printf("Current Directory: %s\n", path);
    else
        printf("Unable to get directory\n");

    printf("Program completed successfully\n");

    return 0;
}
```

OUTPUT SCREENSHOT:

```
chandi@kali:~/Desktop/US36095$ gcc directory.c
chandi@kali:~/Desktop/US36095$ ./directory.c
root@unknown:~# ./directory.c
Current Directory: /home/sabiya
Program completed successfully
root@unknown:~# $
```

4. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

OUTPUTSCREENSHOT:

```
chandi@kali:~/Desktop/US36095$ gcc Histroy.c -o shell
chandi@kali:~/Desktop/US36095$ ./shell
root@unknown:~# ./shell
Enter number of commands: 15

Pipeline:

root@unknown:~# $
```

```
GNU nano 8.7.1 Histroy.c *
#include <stdio.h>
#include <string.h>

int main() {
    char commands[3][50];
    int n;

    printf("Enter number of commands: ");
    scanf("%d", &n);

    getchar();

    for (int i = 0; i < n; i++) {
        printf("Enter command %d: ", i + 1);
        fgets(commands[i], sizeof(commands[i]), stdin);
        commands[i][strcspn(commands[i], "\n")] = '\0';
    }

    printf("\nPipeline:\n");

    for (int i = 0; i < n; i++) {
        printf("%s", commands[i]);

        if (i < n - 1)
            printf(" | ");
    }

    printf("\n");

    return 0;
}
```

-
5. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

Code:

```
chandrika_battina@DESKTOP-US3G0PF
GNU nano 8.7.1 pipe.c *
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    char message[] = "Hello from parent";
    char buffer[100];

    if (pipe(fd) == -1) {
        printf("Pipe creation failed\n");
        return 1;
    }

    write(fd[1], message, strlen(message) + 1);

    read(fd[0], buffer, sizeof(buffer));

    printf("Data received through pipe: %s\n", buffer);

    close(fd[0]);
    close(fd[1]);

    return 0;
}
```

OUTPUT SCREEN SHOT

```
chandr. root@unknown pipe.c
chandr. gcc pipe.c -o shell
root@unknown:~# ./shell
Data received through pipe: Hello from parent
root@unknown
```

6. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

```
root@unknown

GNU nano 8.7.1 redirection.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int input, output;
    char buffer[100];

    input = open("input.txt", O_RDONLY);

    if (input < 0) {
        printf("Input file not found\n");
        return 1;
    }

    dup2(input, STDIN_FILENO);
    close(input);

    output = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output < 0) {
        printf("Output file cannot be created\n");
        return 1;
    }

    dup2(output, STDOUT_FILENO);
    close(output);

    while (fgets(buffer, sizeof(buffer), stdin) != NULL) {
        printf("%s", buffer);
    }

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

OUTPUT SCREENSHOT:

```
chandrka_battina@DESKTOP-US3G0PF [C] ~$ nano redirection.c
chandrka_battina@DESKTOP-US3G0PF [C] ~$ gcc redirection.c -o shell
chandrka_battina@DESKTOP-US3G0PF [C] ~$ ./shell
root@unknown
```

7. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

CODE:

```
root@unknown
GNU nano 8.7.1 append_error.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int file;

    file = open("output.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (file < 0) {
        perror("File error");
        return 1;
    }

    dup2(file, STDOUT_FILENO);
    close(file);

    printf("New data added to the file\n");

    fprintf(stderr, "This is an error message\n");

    return 0;
}
```

OUTPUT SCREENSHOT:

```
chandrka battina@DESKTOP-US3G0PF ~$ nano append_error.c
chanc root@unknown c append_error.c -o shell
root@unknown ./shell
This is an error message
root@unknown:~#
```

-
8. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

CODE:

```
root@unknown

GNU nano 8.7.1 combined.c *
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int file;

    printf("Starting combined redirection...\n");

    file = open("result.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (file < 0) {
        perror("File error");
        return 1;
    }

    write(file, "Command 1: Listing files\n", 25);
    write(file, "Command 2: Processing output\n", 30);
    write(file, "Pipeline and redirection completed successfully\n", 48);

    close(file);

    printf("Output redirected to result.txt\n");
    printf("Combined redirection completed successfully\n");

    return 0;
}
```

OUTPUTSCREENSHOT:

```
chandrika.battina@DESKTOP-HIS3G0PF ~$ nano combined.c
chand root@unknown combined.c -o shell
root@unknown:~# ./shell
Starting combined redirection...
Output redirected to result.txt
Combined redirection completed successfully
root@unknown:~#
```

9. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

Code:


```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    printf("Starting background job...\n");

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Background process started. PID = %d\n", getpid());
        sleep(5);
        printf("Background process completed.\n");
        return 0;
    }

    printf("Job started in background. PID = %d\n", pid);
    printf("Parent process is free to continue.\n");

    sleep(1);

    if (waitpid(pid, &status, WNOHANG) == 0) {
        printf("Job is still running...\n");
    }

    sleep(5);

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Background job completed successfully.\n");
    }

    return 0;
}
```

OUTPUT SCREENSHOT:

```
chandrika.battina@DESKTOP-US3G0PF $ nano background.c
chandrika.battina@DESKTOP-US3G0PF $ gcc background.c -o shell
chandrika.battina@DESKTOP-US3G0PF $ ./shell
Starting background job...
Job started in background. PID = 1131
Parent process is free to continue.
Background process started. PID = 1131
Job is still running...
Background process completed.
Background job completed successfully.
root@unknown:~#
```

10. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    printf("Starting job...\n");

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child process running. PID = %d\n", getpid());
        sleep(5);
        printf("Child process completed.\n");
        return 0;
    }

    printf("\nActive Jobs:\n");
    printf("[1] PID = %d Running\n", pid);

    sleep(2);

    if (waitpid(pid, &status, WNOHANG) == 0) {
        printf("\nJob is still running.\n");
        printf("Switching job to foreground...\n");

        waitpid(pid, &status, 0);

        printf("Foreground job completed.\n");
    }

    return 0;
}
```

OUTPUT SCREENSHOT:

```

chandrika_battina@DESKTOP-US3G0PF:~$ gcc jobs.c
chandrika_battina@DESKTOP-US3G0PF:~$ ./shell
Starting job...

Active Jobs:
[1] PID = 1183  Running
Child process running. PID = 1183

Job is still running.
Switching job to foreground...
Child process completed.
Foreground job completed.
root@unknown:~#

```

11. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

CODE:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child process started. PID = %d\n", getpid());
        raise(SIGSTOP);

        printf("Child process resumed\n");
        return 0;
    }

    sleep(1);

    printf("Child process is stopped\n");
    printf("Sending SIGCONT signal...\n");

    kill(pid, SIGCONT);

    waitpid(pid, NULL, 0);

    printf("Child process completed\n");

    return 0;
}

```

OUTPUT SCREEN SHOT:

```

chandrka battina@DESKTOP-US3G0PF ...$ nano signal.c
chandri root@unknown:~$ gcc signal.c -o shell
root@unknown:~# ~$ ./shell
Child process started. PID = 1231
Child process is stopped
Sending SIGCONT signal...
Child process resumed
Child process completed
root@unknown:~#

```

12. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

CODE:

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig) {
    printf("\nSIGINT received\n");
    printf("Process is protected from termination\n");
}

int main() {
    signal(SIGINT, handle_signal);

    printf("Program is running...\n");
    printf("Press Ctrl+C to test SIGINT\n");

    while (1) {
        sleep(1);
    }

    return 0;
}

```

OUTPUT SCREENSHOT:

```

Child process completed
chandri root@unknown:~$ gcc sigint.c -o shell
chandri ~$ ./shell
root@unknown:~# ~$ ./shell
Program is running...
Press Ctrl+C to test SIGINT
^C
SIGINT received
Process is protected from termination
^C
SIGINT received
Process is protected from termination
^Z
[1]+  Stopped                  ./shell
root@unknown:~# ~$ |

```

-
13. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

CODE:

```
#include <unistd.h>

void handle_stop(int sig) {
    printf("\nSIGTSTP received\n");
    printf("Process will continue running\n");
}

int main() {
    signal(SIGTSTP, handle_stop);

    printf("Process started\n");
    printf("PID: %d\n", getpid());
    printf("Press Ctrl+Z to test SIGTSTP\n");

    while (1) {
        sleep(2);
        printf("Process is running...\n");
    }

    return 0;
}
```

OUTPUT SCREENSHOT:

```
chandr@battina@DESKTOP-JIS3G0PE: ~$ cat sigtstp.c
chandr@battina@DESKTOP-JIS3G0PE: ~$ gcc sigtstp.c
root@unknown:~# ./sigtstp.c
Process started
PID: 1341
Press Ctrl+Z to test SIGTSTP
Process is running...
Process is running...
Process is running...
^Z
SIGTSTP received
Process will continue running
Process is running...
```

-
14. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *file;
    char data[100];

    printf("Starting OSSP module integration...\n");

    file = fopen("input.txt", "r");

    if (file == NULL) {
        printf("Error: input.txt not found\n");
        printf("Program recovered gracefully\n");
        return 1;
    }

    if (fgets(data, sizeof(data), file) != NULL) {
        printf("File data: %s", data);
    }

    fclose(file);

    printf("Modules executed successfully\n");
    printf("End-to-end execution completed\n");

    return 0;
}
```

OUTPUT SCREENSHOT:

```
chandr@DESKTOP-U53G0PF:~$ nano integration.c
chandr@DESKTOP-U53G0PF:~$ gcc integration.c -o shell
chandr@DESKTOP-U53G0PF:~$ ./shell
Starting OSSP module integration...
File data: I am 2520030502 - Shaik Sabiya Tabassum
Modules executed successfully
End-to-end execution completed
root@unknown:~#
```

-
15. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

CODE:

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;

    ptr = (int *)malloc(5 * sizeof(int));

    if (ptr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        ptr[i] = i + 1;
    }

    printf("Allocated values:\n");

    for (int i = 0; i < 5; i++) {
        printf("%d ", ptr[i]);
    }

    printf("\n");

    free(ptr);

    printf("Memory released successfully\n");

    return 0;
}

```

OUTPUT SCREENSHOT:

```

chandrika@DESKTOP-US3G0PF:~/C++$ gcc memory.c
chandrika@DESKTOP-US3G0PF:~/C++$ ./shell
Allocated values:
1 2 3 4 5
Memory released successfully
chandrika@DESKTOP-US3G0PF:~/C++$

```

16. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t p1, p2, p3;

    printf("Starting multiple jobs...\n");

    p1 = fork();

    if (p1 == 0) {
        printf("Job 1 running. PID = %d\n", getpid());
        sleep(2);
        printf("Job 1 completed\n");
        return 0;
    }

    p2 = fork();

    if (p2 == 0) {
        printf("Job 2 running. PID = %d\n", getpid());
        sleep(3);
        printf("Job 2 completed\n");
        return 0;
    }

    p3 = fork();

    if (p3 == 0) {
        printf("Job 3 running. PID = %d\n", getpid());
        sleep(1);
        printf("Job 3 completed\n");
        return 0;
    }

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
    waitpid(p3, NULL, 0);

    printf("All jobs completed successfully\n");

    return 0;
}
```

OUTPUT SCREENSHOT:

```
chandrika_battina@DESKTOP-US3G0PF ~$ nano performance.c
chandrika_battina@DESKTOP-US3G0PF ~$ gcc performance.c -o shell
chandrika_battina@DESKTOP-US3G0PF ~$ ./shell
Starting multiple jobs...
Job 1 running. PID = 1481
Job 2 running. PID = 1482
Job 3 running. PID = 1483
Job 3 completed
Job 1 completed
Job 2 completed
All jobs completed successfully
root@unknown:~#
```


17. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("OSSP Shell Project Demo\n");
    printf("-----\n");

    printf("1. Process creation\n");

    pid = fork();

    if (pid < 0) {
        printf("Process creation failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("2. Child process created\n");
        printf("3. Child process executing command\n");
        execl("/bin/ls", "ls", NULL);

        printf("Execution failed\n");
        return 1;
    }

    waitpid(pid, NULL, 0);

    printf("4. Parent process completed\n");
    printf("5. Project demonstration completed successfully\n");

    return 0;
}
```

OUTPUT SCREENSHOT:

```

sabiya@LAPTOP-BOV34HDC:~$ nano demo.c
chandr root@unknown .c -o shell
chandr

OSSP Shell Project Demo
-----
1. Process creation
2. Child process created
3. Child process executing command
52Commands      a.out      malloc.c      processspace.c
FCFS.c          address.c  memory.c      redirection.c
Fork.c          addressTranslation.c  output.txt    result.txt
Fork2.c         append_error.c  parser        sabi.c
Histroy.c       background.c  parser.c      sabiya
OSLab1.c        cd.c        particalonetwo.c  segmentationFault.c
RoundRobyn.c    combined.c   performance.c  shell
SJFNonPreemptive.c  demandpagging.c  pipe.c        sigint.c
WEEK1          demo.c      pipes         signal.c
WEEK1-6.c       directory.c  pipes.c       signals.c
WEEK1.2.c       dynamic.c   practicalonetwo.c  sigtstp.c
WEEK1.3.c       escape      practicaltwo.c  singlequote
WEEK1.4.c       escape.c    priority.c     singlequote.c
WEEK1.5.c       excute.c    process.c      skill2.c
WEEK1.6.c       filecopy    processconcept1.c  tokenizer.c
WEEK1.7.c       history.c   processconcept5.c  variable.c
WEEK1.8.c       input.txt   processconcepts.c  virtual
WEEK1.8.c.save  integration.c  processconcepts1.c  virtual.c
WEEK1.9.c       jobs.c      processconcepts2.c  waitpid
WEEK1.c         keyboard.c  processconcepts3.c  waitpid.c
WEEK2.c         mainloop.c  processconcepts4.c

4. Parent process completed
5. Project demonstration completed successfully
root@unknown:~#

```